Projeto Integrador Assistente de Desenvolvimento de Aplicativos Computacionais II

Curso: Técnico em Informática

Turma: INF 4678

Professor: Rodrigo Michel

Stack: Python + Django

Equipes: 3 integrantes

Datas: Entrega: 27/02 | Apresentação: 02/03

Escopo do Projeto Integrador

Briefing do cliente:

A ***TechVentures***, uma *holding* de investimentos em tecnologia, acabou de adquirir a sua *Software House*. A diretoria precisa validar a capacidade técnica da sua equipe urgentemente.

Para isso, eles apresentaram **4 demandas** que estão paradas no backlog da empresa. A sua equipe tem autonomia para **escolher apenas UMA** dessas missões para desenvolver o MVP (Produto Mínimo Viável) e entregar funcionando em **27/02.**

A infraestrutura da *TechVentures* é 100% baseada em **Python**, portanto, o uso do framework **Django** é obrigatório.

As Opções de Projeto (Escolha a sua Missão):

Opção 1: Sistema de Gestão de Ativos (Inventário)

"Estamos perdendo equipamentos nos nossos escritórios."

- **O Problema:** A empresa não sabe quem está com qual notebook, monitor ou celular corporativo.
- **O Desafio:** Criar um sistema onde o **Admin** cadastra os equipamentos (patrimônio, marca, modelo) e o **Colaborador (Cliente)** pode visualizar o catálogo e solicitar a retirada de um item.

Opção 2: Agenda Corporativa (Reserva de Salas)

"Nossas reuniões estão uma bagunça, ninguém sabe qual sala está livre."

- **O Problema:** Conflito de horários no uso das salas de reunião e laboratórios.
- **O Desafio:** Criar um sistema onde o **Admin** cadastra as salas disponíveis (capacidade, recursos) e o **Colaborador (Cliente)** visualiza um calendário/lista e realiza o agendamento de um horário.

Opção 3: Portal de Conhecimento (Base de Dúvidas)

"O suporte técnico responde sempre as mesmas perguntas todo dia."

- **O Problema:** Perda de tempo da equipe de TI respondendo dúvidas repetitivas.
- **O Desafio:** Criar uma base de conhecimento onde o **Admin** posta artigos, tutoriais e soluções de erros, e o **Usuário (Cliente)** pode pesquisar por palavras-chave, visualizar a solução e dar um feedback se funcionou ou não.

Opção 4: Open Innovation (Carta Branca)

"Surpreenda a diretoria."

- **O Cenário:** Sua equipe identificou uma falha ou oportunidade no mercado e acredita que pode resolvê-la com Python/Django.
- **O Desafio:** Vocês propõem o tema.
- **Condição:** A ideia precisa ser aprovada na **Aula 1**.

Requisitos Técnicos (Obrigatórios)

1. **Tecnologia Base:**
    a. Linguagem: Python.
    b. Framework Web: Django (Padrão MVT - Model, View, Template).
    c. Banco de Dados: SQLite (padrão do projeto), MySql ou PostgreSQL.
2. **Segurança e Acesso: O sistema deve ter Login funcional (Django Auth).**
    a. Visão Admin (Back-office): O administrador deve ter painéis para criar, editar e excluir registros (CRUD) usando o Django Admin.
    b. Visão Cliente (Front-end): usuário final não acessa o painel admin, mas sim a interface desenvolvida com Django Templates (HTML/CSS).
3. **Modelagem de Dados (Models):**
    a. Uso do Django ORM para criação das tabelas.
    b. O código deve conter pelo menos 2 modelos (tabelas) relacionados (OneToMany ou ManyToMany).
4. **Esboço de Telas:**
    a. Wireframes feitos no Figma.com, Wireframe.cc (ou similar) com as principais telas do sistema.
5. **Gerenciamento de Projeto:**
    a. Cada grupo deve usar uma ferramenta de gerenciamento para organizar o trabalho, rastrear tarefas e visibilidade de progresso.
    b. Sugestões:
        i. Trello: quadros com listas (Backlog, Andamento, Concluído).
        ii. Jira: (Kanban ou Scrum Board)
6. **Versionamento:**
    a. Repositório GitHub com .gitignore configurado corretamente.
    b. Deve conter commits de todos os integrantes do grupo.
    c. README explicativo do projeto.

Entregas Mínimas (MVP para passar na disciplina)

- Ambiente virtual (venv) configurado e **requirements.txt** gerado.
- Models criados e migrados (makemigrations / migrate).
- Django Admin funcional
- Pelo menos três Views (FBV ou CBV) renderizando Template.
- Uso da hierarquia de templates
- Uso de arquivos estáticos (CSS/Imagens) ({% static %}).
- Formulário funcional no lado do cliente (para busca ou cadastro simples).
- Interface navegável (mesmo que simples).
- Documentação no README com:
    - Descrição do projeto
    - Tecnologias usadas
    - Lista de integrantes e papéis
    - Fluxo de contribuição

Prazos e Organização

- 1ª Aula: Definição do tema, papéis no grupo, criação do repositório, Configuração do ambiente e primeira versão da *landing page* (Wireframe).
- 2ª Aula: Finalização dos *wireframes* e início do desenvolvimento.
- 3ª Aula: Funcionalidades principais implementadas e testadas.
- 4ª Aula: Ajustes finais, testes, documentação e entrega.

Critérios de Avaliação

- Cumprimento dos requisitos mínimos.
- Qualidade do código e organização do projeto no GitHub.
- Qualidade dos wireframes.
- Funcionalidade real do sistema.
- Participação de todos os integrantes.
- Apresentação do Projeto.

Grupos

GRUPO 1
MARCELO FLÔRES BALLIN,
MIGUEL NEHEMIAS POLEO DELGADO,
KAINÃ BRUM HABEKOST

GRUPO 2
DANIEL GUSTAVO PETTRY MAUS,
GABRIEL AUGUSTO KAPPENBERG,
LORENZZO ANTONIO PATATT

GRUPO 3
PIETRO SAMUEL DOS SANTOS PUNTEL,
ERICK BORGES RODRIGUES,
KAUÃ LONGARAY KELLER

GRUPO 4
ANA CAROLINA MACHADO
MILLENA KAPPAUN
CHARLES KUNDE VITALE

GRUPO 5
YURI PARANHOS
MARCELLO SILVEIRA MACHADO
ERICK FELIPE STAUB